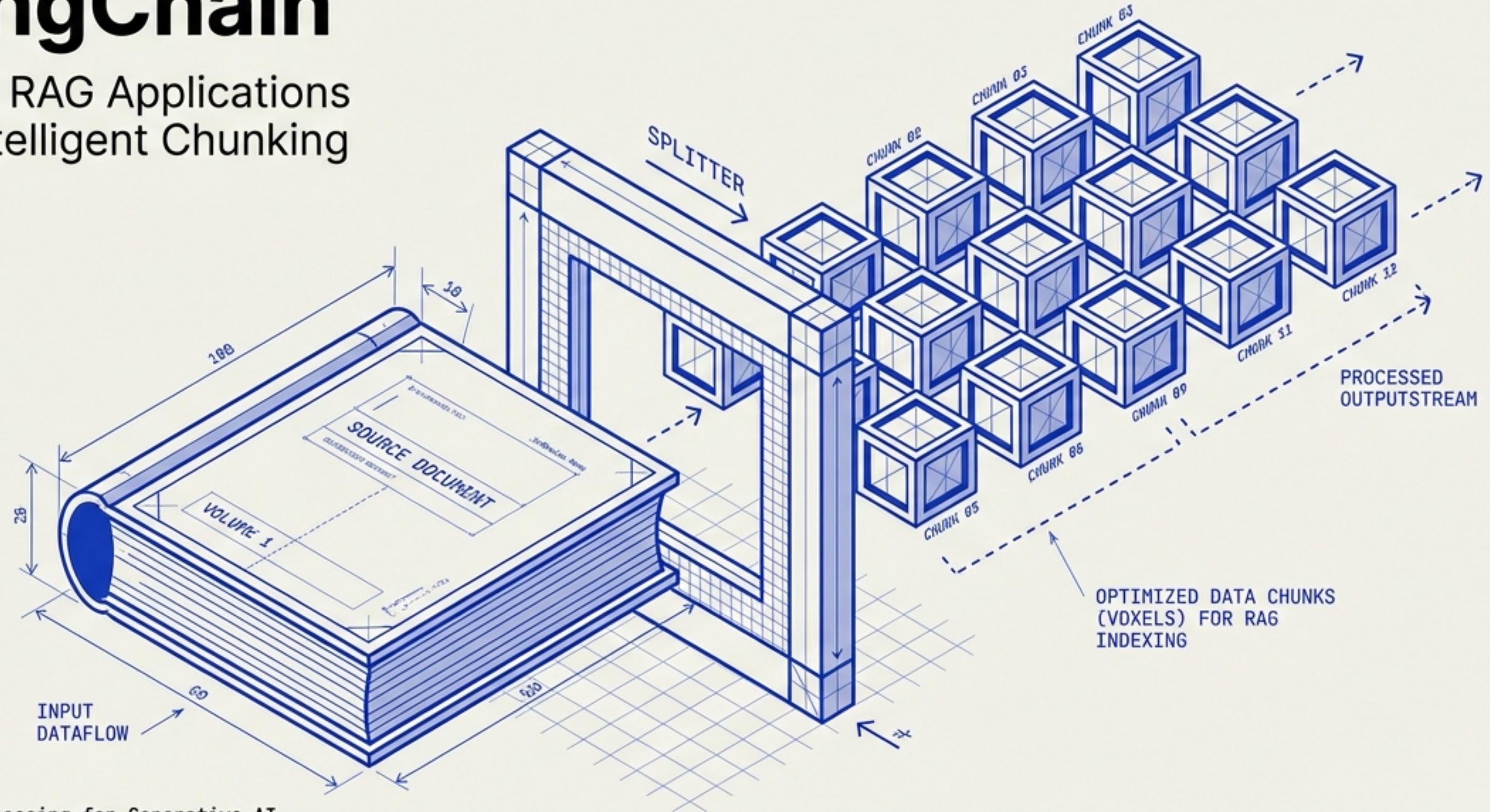


Mastering Text Splitters in LangChain

Optimising RAG Applications through Intelligent Chunking



The Context Window Bottleneck

The Constraint

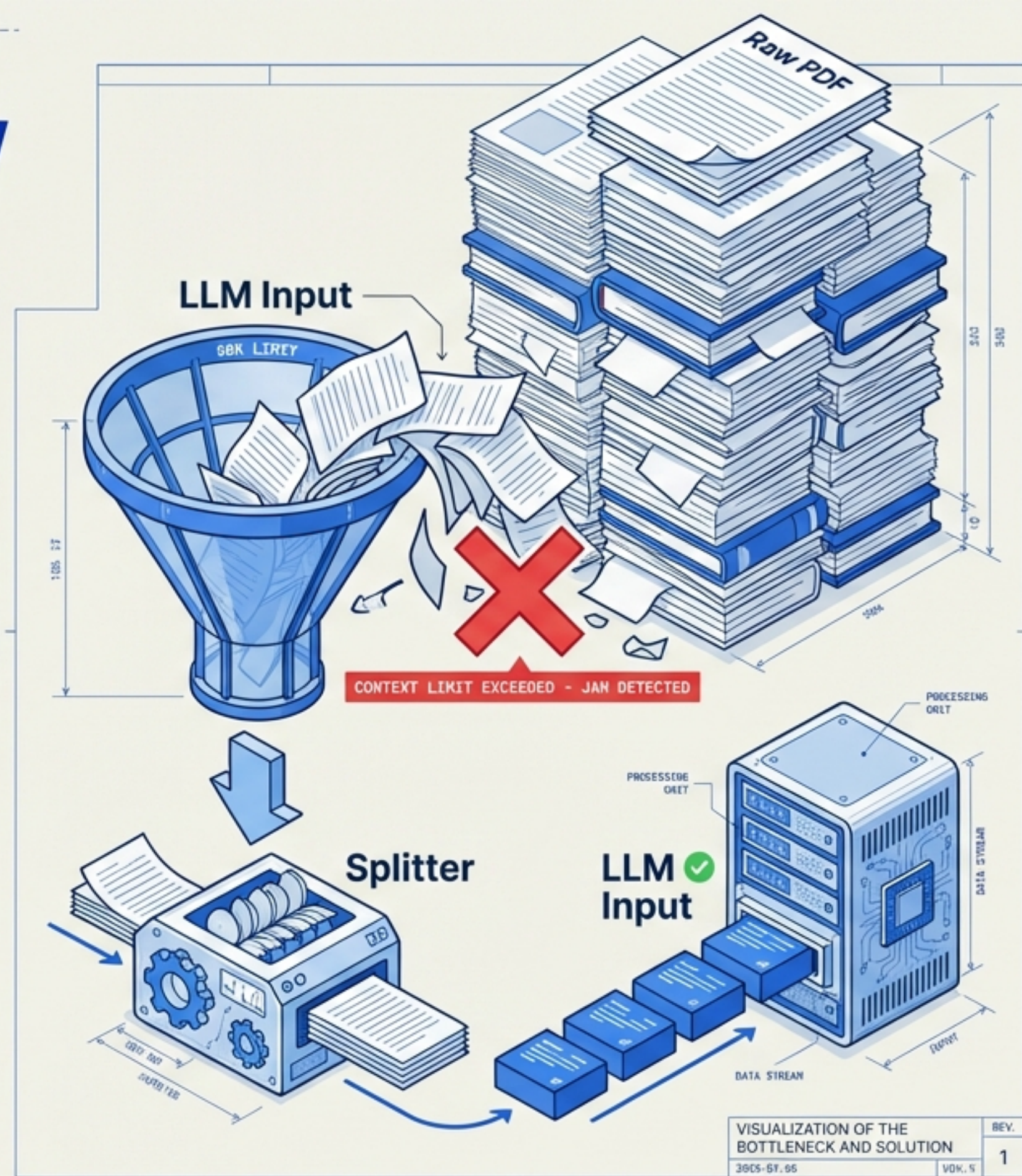
- ❑ Large Language Models have a hard context limit (e.g., 50k tokens).

■ The Reality

Real-world assets (PDFs, Books)
often exceed 100k+ words.

■ The Solution

Text Splitting bridges raw data and LLM comprehension.

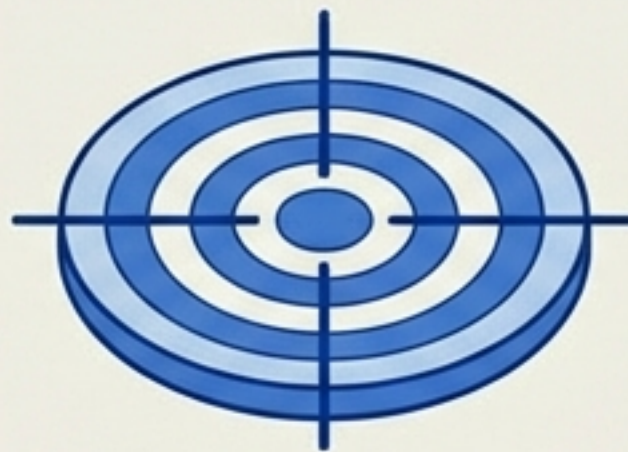


Three Strategic Advantages of Splitting



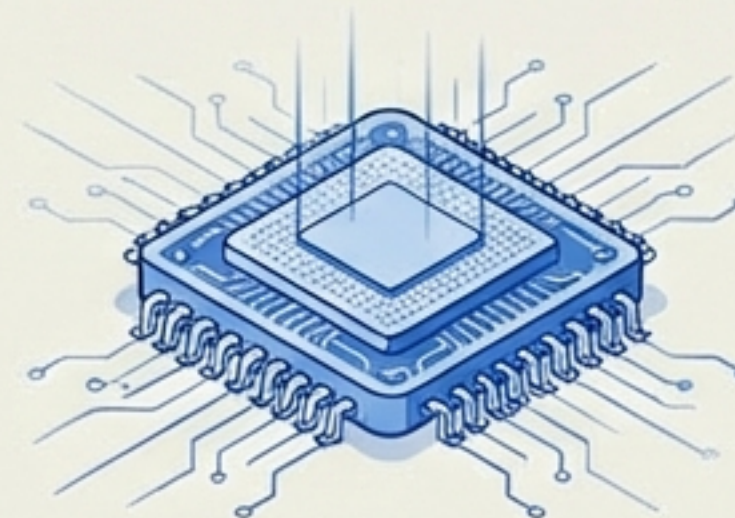
Overcoming Limits

Bypasses input size constraints of Embedding and Language models, allowing massive documents to be processed.



Quality & Precision

Prevents 'Context Drift' and hallucinations. Focused chunks improve Semantic Search accuracy by keeping topics distinct.

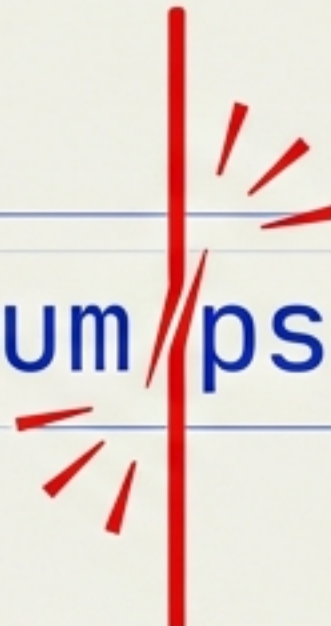


Computational Efficiency

Lowers memory usage (RAM) and enables parallel execution for faster processing speeds.

The Naïve Approach: Length-Based Splitting

The quick brown fox jumps over the lazy dog.



Inter

Mechanism:

The simplest method. Predetermines a strict size (e.g., 100 chars) and cuts blindly.

LangChain Class: "CharacterTextSplitter"

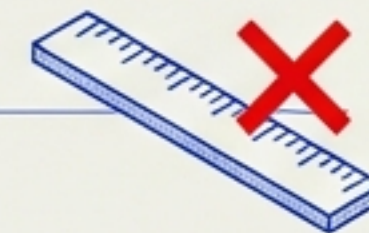


Inter

The Flaw:

Ignores linguistic structure.

Result: Words severed mid-character.
Sentences decapitated. Context lost.



Preserving Context with 'Chunk Overlap'

Chunk A

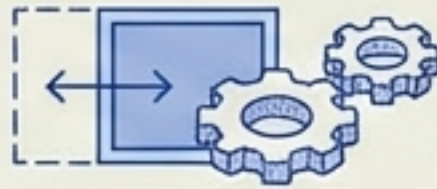


Chunk B



The Problem:

Hard cuts sever meaning at the boundary.



The Solution:

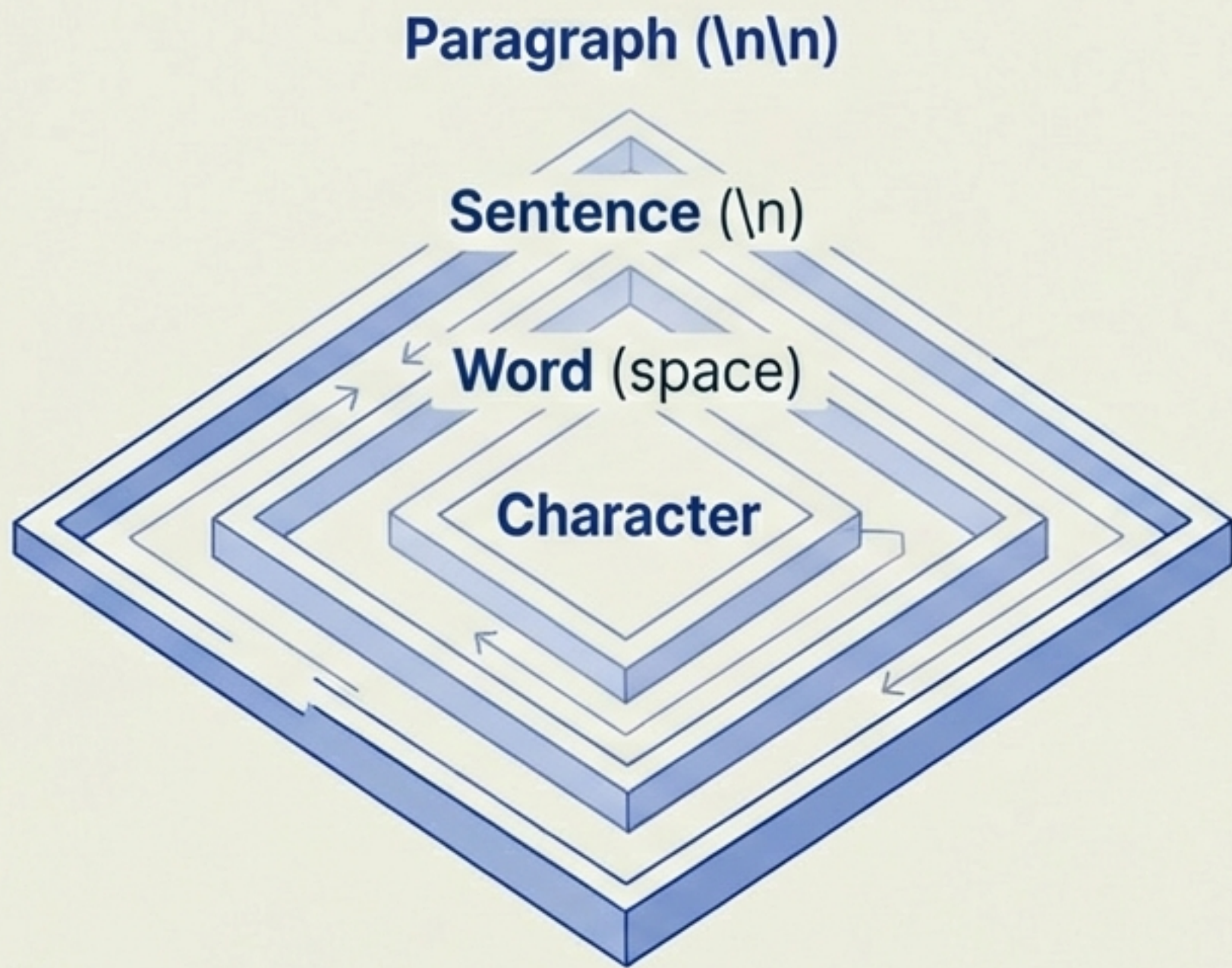
A sliding window mechanism. The end of Chunk A repeats as the start of Chunk B.



Best Practice:

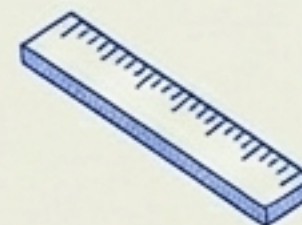
10–20% overlap is standard for RAG applications to maintain narrative flow.

The Gold Standard: Recursive Character Text Splitting



Philosophy:

Respect the linguistic hierarchy.



Mechanism:

Attempts to split by paragraph first. If the chunk is too large, it 'backs off' to sentences, then words.

LangChain Class:

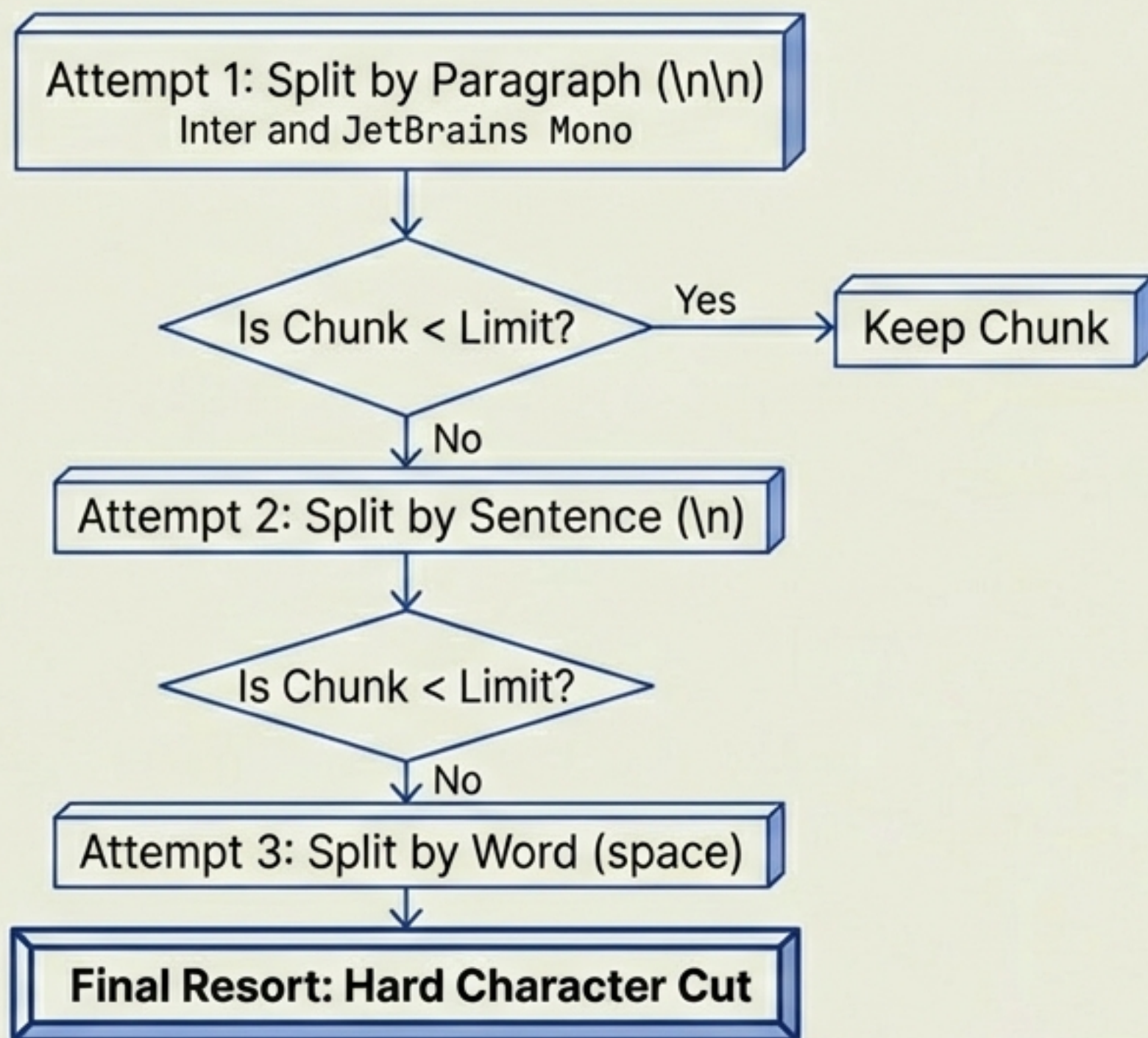
`RecursiveCharacterTextSplitter`



Why it Wins:

Keeps related text together. Paragraphs remain intact unless absolutely necessary.

The Recursive 'Back-off' Strategy



Structure Beyond Prose: Code & Markdown



```
class DocumentProcessor:

    def __init__(self, file_path):
        self.file_path = file_path

    def get_title(self):
        # Returns document title
        return self.file_path.split('/')[-1]

    def parse_content(self):
        pass
```

Python: Splits by class / def



Section Title

This is a paragraph within the section. It contains important information related to the header.

- List item 1
- List item 2

Subsection

More details about the subsection go here.

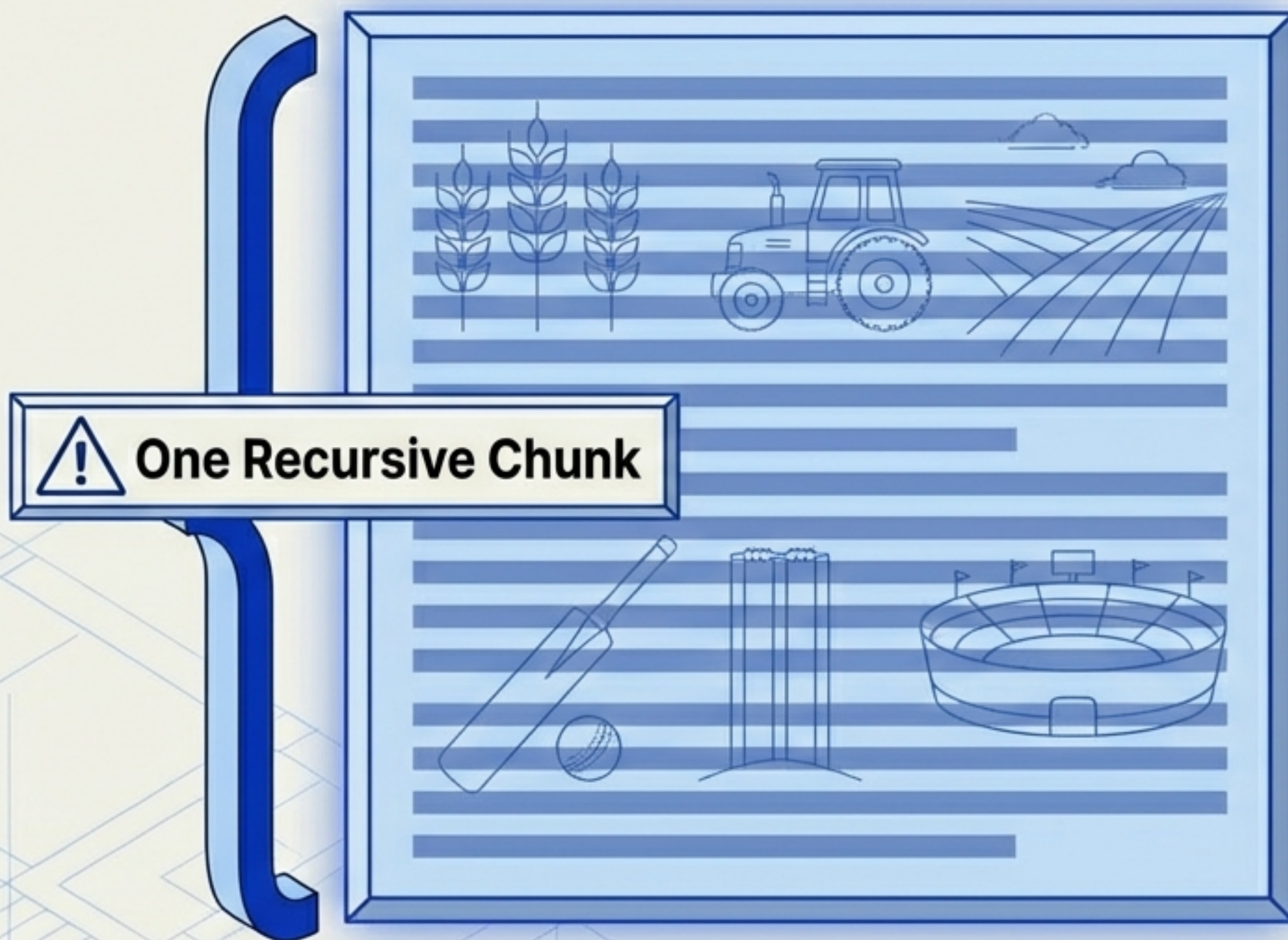
Markdown: Splits by Headers (#, ##)

Challenge: Code/Markdown rely on logical blocks, not paragraphs.

Solution: Language-Specific Separators keep functions and sections intact.

Implementation:
'RecursiveCharacterTextSplitter
.from_language(...)'

The Semantic Blind Spot



The Scenario:

A single document switching topics abruptly (e.g., Agriculture -> Cricket).

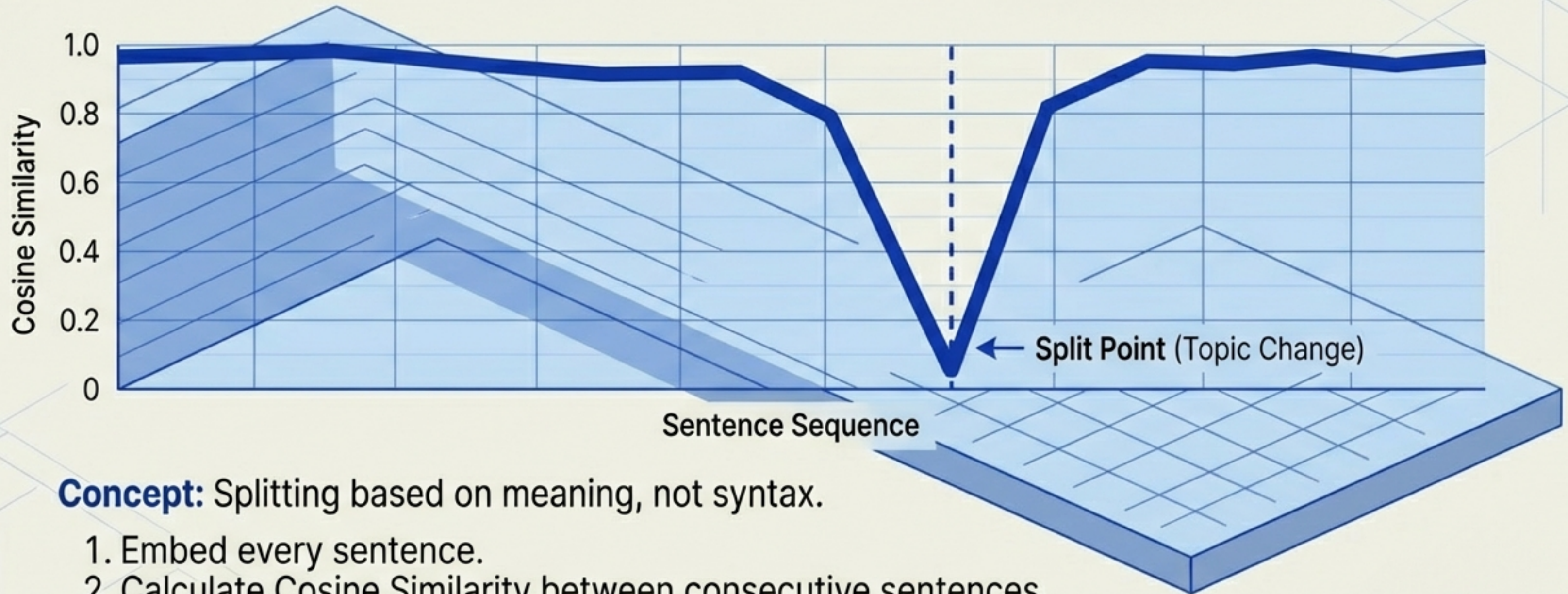
The Failure:

Structural splitters keep this paragraph together.

The Consequence:

Vector Dilution. The embedding becomes a confusing mix of 'Farming' and 'Sports', degrading retrieval accuracy.

The Frontier: Semantic Chunking

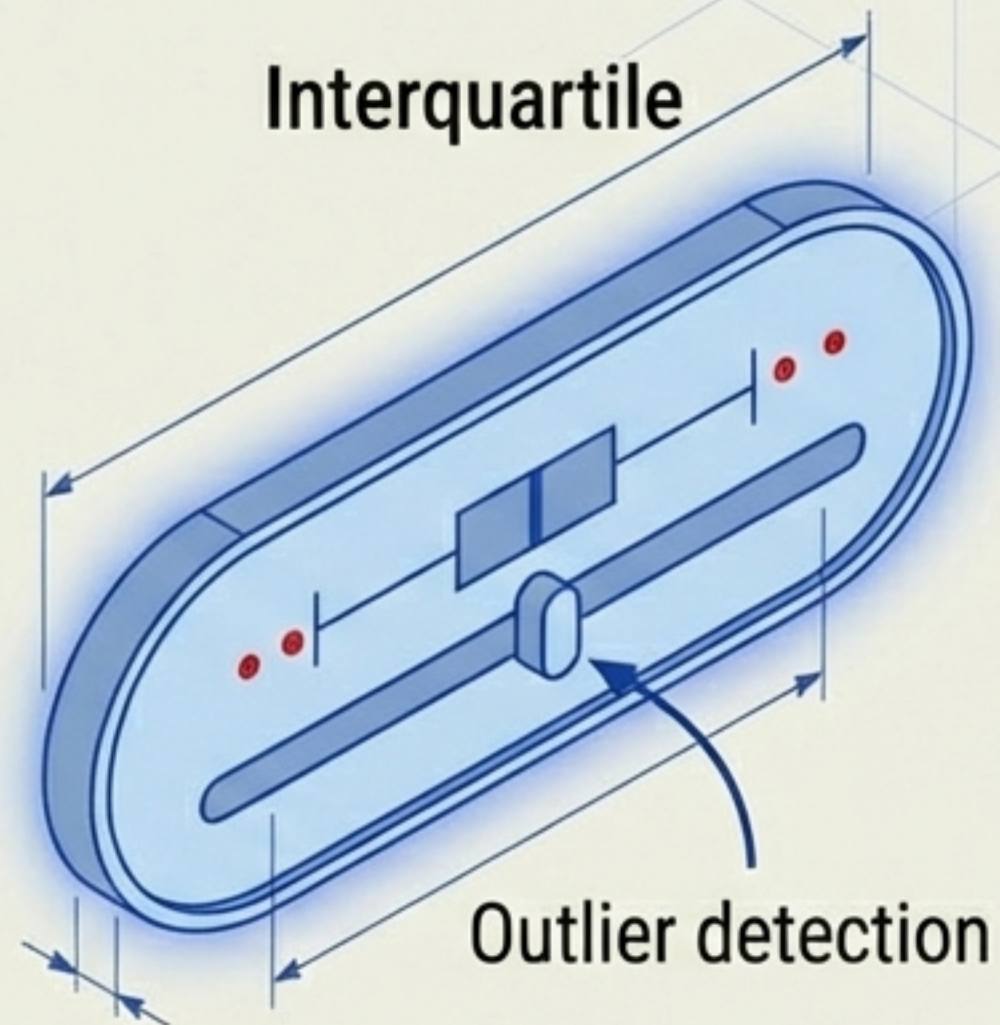
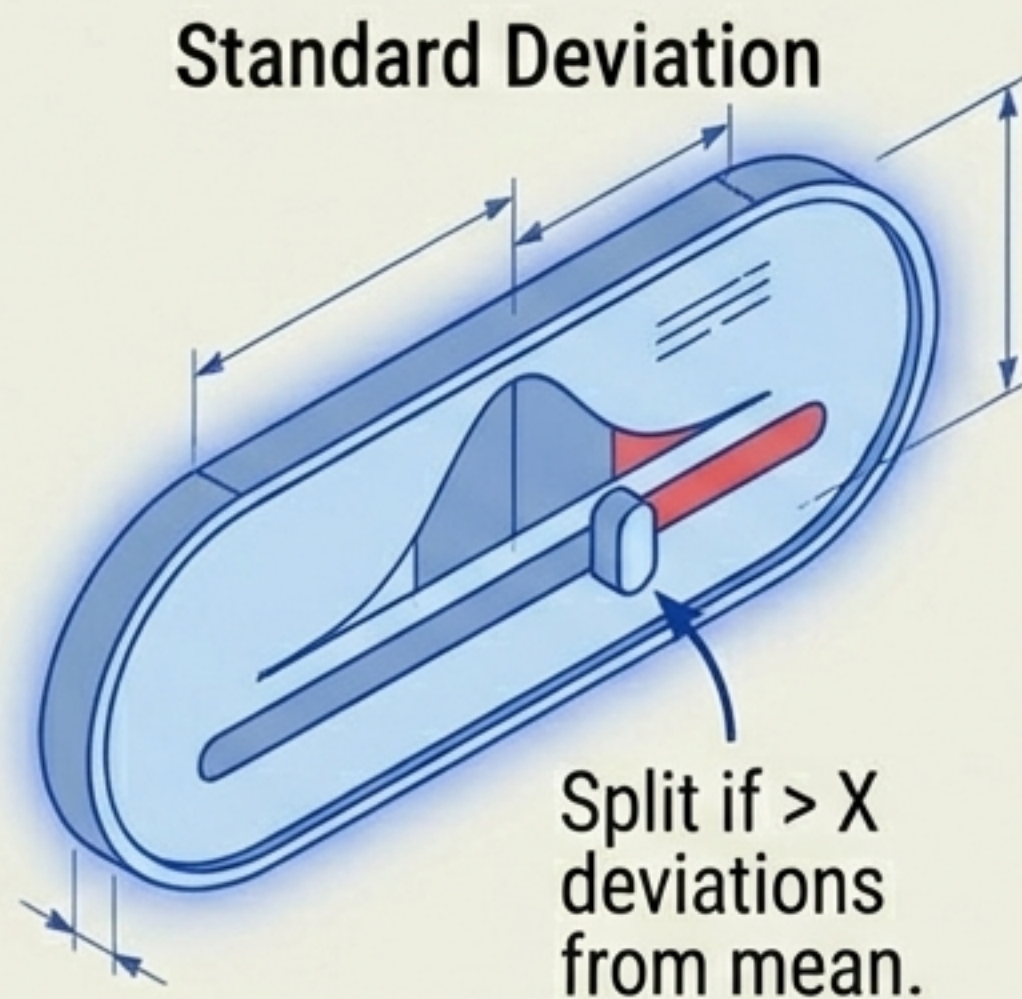
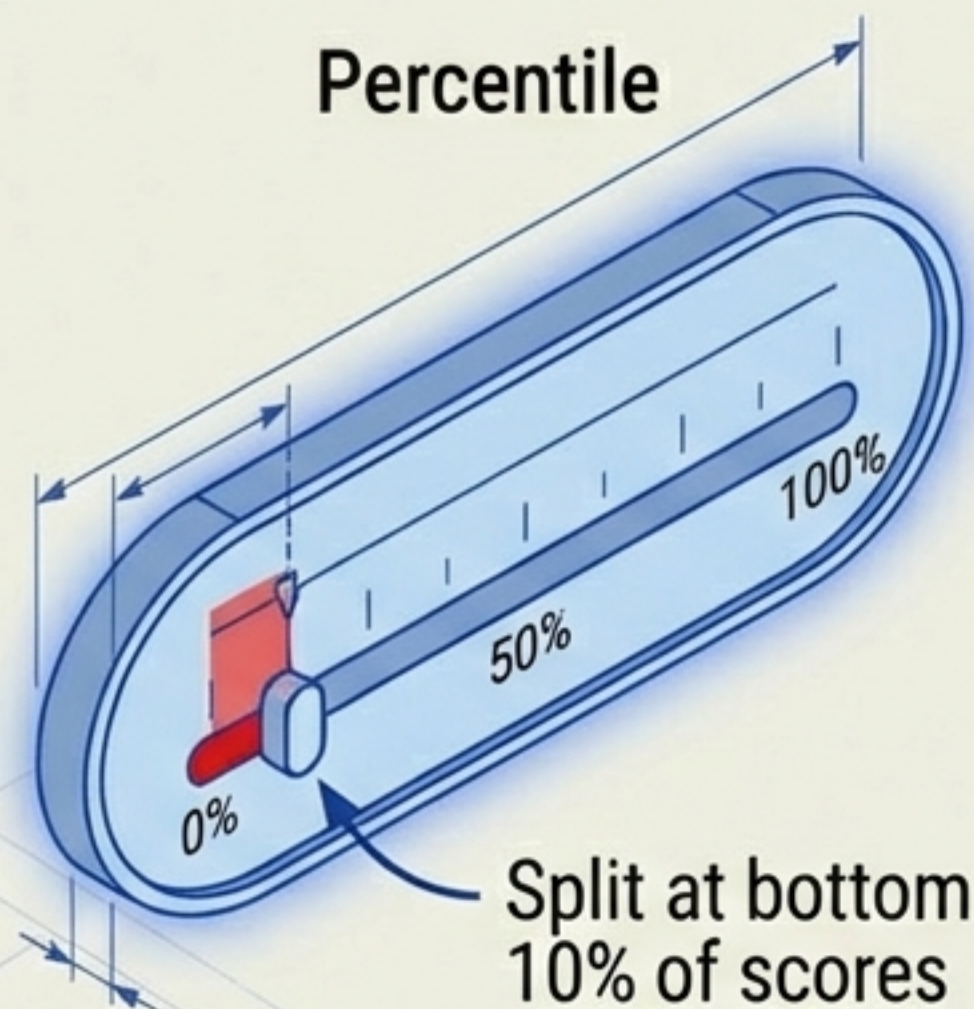


Concept: Splitting based on meaning, not syntax.

1. Embed every sentence.
2. Calculate Cosine Similarity between consecutive sentences.
3. Identify drops in similarity to detect topic shifts.

LangChain Class: `'SemanticChunker'` (Experimental)

Configuring the Semantic Threshold



The Break Point: Defines how different sentences must be to trigger a split.

Note: Computationally expensive (requires embedding every sentence).

Implementation Pattern in LangChain

Import Class

1. Import

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

Configure Parameters

2. Initialise

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50  
)
```

Execute Split

3. Load & Process

```
docs = loader.load()
```

4. Split

```
final_chunks = splitter.split_documents(docs)
```


Summary: Choosing the Right Splitter

Use Case	Recommended Splitter	Why?
General Prose / Articles	 Recursive Character (Default) JetBrains Mono	Balances structure and context. The industry standard.
Code (Python/JS)	 Language-Specific JetBrains Mono	Preserves functional logic (classes/functions).
Markdown / Notion	 Markdown Header JetBrains Mono	Respects document hierarchy and sections.
High-Precision RAG	 Semantic Chunker JetBrains Mono	Ensures topic purity. (Experimental / High Compute).
Simple Prototyping	 Character Splitter JetBrains Mono	Fast, but prone to cutting words.

The Foundation of RAG Intelligence



Text splitting is not merely formatting. It is the critical **pre-processing** layer that defines the **retrieval quality** of your application.

The Sweet Spot Recommendation



Start with Recursive Character Splitting.



Size: 500-1000 tokens.



Overlap: 10-20%.



Iterate based on retrieval metrics.